MODELS OF COMPUTATION (BACSE201)

PROJECT REPORT

AUTOSEC

Counterexample-Driven Automata and Syntax Demo

Team Members

Bhoumik Singh Rajput | 25BYB0120

Vihaan Jinesh Jain | 25BYB0130

Faculty

Dr. Dharmendra Singh Rajput

School of Computer Science Engineering and Information Systems

Vellore Institute of Technology, Vellore

Academic Year 2026–2027

Abstract

Pattern occurrence and syntactic validity are distinct properties of an input. AUTOSEC is a browser-local teaching application that evaluates these properties independently and explains differences between regular rules through shortest counterexamples. A TypeScript engine parses supported regular expressions, constructs Thompson ε-nondeterministic finite automata, computes ε-closures, performs subset construction, and minimises the resulting deterministic machines. A second pipeline tokenises a restricted nested command language and performs predictive parsing using calculated FIRST/FOLLOW sets and a single-lookahead parsing table. Breadth-first search over a product automaton returns either language equivalence or a shortest string accepted by exactly one rule. The interface presents two tasks, Check text and Compare rules, with optional execution traces. Eight independently specified languages were evaluated on all 127 binary strings of length at most six. Genuine application screenshots illustrate valid nested syntax, a missing flag value, and the distinguishing string `--a`. The contribution is the integration of independent pattern and syntax verdicts with a reproducible counterexample workflow. The evaluation establishes observed implementation behaviour; educational effectiveness remains a question for a future user study.

Index Terms: formal languages, finite automata, regular expressions, predictive parsing, context-free grammar, shortest counterexample, educational software.

Report organisation

I. Introduction and Related Work	3
II. Formal Definitions and Recognition Semantics	4
III. Methodology and System Architecture	5
IV. Regular-Language Construction	6
V. Lexical Analysis and Predictive Parsing	7
VI. Shortest-Counterexample Method	8
VII. Actual Application Screenshots	9
VIII. Evaluation and Observed Results	10
IX. Discussion, Limitations and Conclusion	11
References	12

Project scope

- **Course basis:** Regular-language recognition, nested syntax, and language comparison share one command-like example. Reference [1] provides the formal-language and automata background.
- **Interaction:** Users inspect a result, make a small change, and examine why acceptance changes. The application analyses inputs as text.
- **Presentation:** The interface concentrates on two tasks; this report supplies the definitions, diagrams, methodology, evaluation, and limitations.

I. Introduction and Related Work

A. Problem statement

The strings `inspect --tag sample` and `inspect --tag` both contain a flag-shaped substring. Their syntax differs because every flag requires a value.

- **Pattern occurrence:** Both strings contain `--tag`, so the regular pattern matches both inputs.
- **Syntactic validity:** Only the first string supplies the required value. The parser rejects the second at end-of-input.
- **Nested structure:** Parenthesised command groups require the parser to retain expected closing delimiters. A local flag match does not establish this structural property.

Recent educational work explains regex-to-automaton transformations [2], nondeterministic rejection [3], and machine/stack behaviour [4]. Recipe-based diagnostics support regex development [5]. Table I compares their documented emphasis with AUTOSEC's design; it is a qualitative comparison rather than a system benchmark.

TABLE I. DOCUMENTED EMPHASIS AND AUTOSEC DESIGN POSITION

Work	Documented emphasis	AUTOSEC emphasis
FSM visualisation [2]-[4]	Transformations, computation graphs and machine/stack execution.	A command with two verdicts, then a shortest rule disagreement.
Recipe support [5]	Regex design process and targeted diagnostic feedback.	A small set of prepared contrasts and optional computed steps.
automata [6]	Python API, simulation, conversion, language operations and equivalence.	Direct browser forms without coding an automata API.
AALpy [7]	Active model learning, testing and equivalence oracles.	Exact product search between two supplied regex-derived DFAs.

B. Distinction and objectives

AUTOSEC combines **two independent verdicts on the same command** with **a shortest witness between two explicit rules** in a two-task interface.

- **Design contribution:** Inspect a command, identify a syntax failure, change a rule, and explain the disagreement through computed traces.
- **Established foundations:** The automata library supplies transformations and language operations [6]; AALpy uses counterexamples in model learning [7]. AUTOSEC's contribution concerns integration and presentation.
- **Implementation objectives:** Compute verdicts from explicit machines, preserve token offsets, explain the first syntax failure, and reproduce rule disagreements.
- **Evaluation boundary:** Independent predicates and recorded examples check implementation behaviour. Comparative accuracy, usability, and speed require a separate study.

II. Formal Definitions and Recognition Semantics

A. Symbols, languages and finite-state machines

An **alphabet** Σ is a finite set of symbols. A **string** w is a finite sequence over Σ; ε denotes the empty string and $|w|$ its length. A **language** is a set of strings, $L \subseteq \Sigma^*$.

- **Regular expression:** A language description formed through literals, concatenation, union, and repetition. In the supported syntax, `r*` denotes zero or more repetitions, `r+` one or more, and `r?` zero or one.
- **Deterministic finite automaton (DFA):** $M = (Q, \Sigma, \delta, q_0, F)$, where Q is a finite state set, $\delta: Q \times \Sigma \to Q$ is the transition function, $q_0 \in Q$ is the initial state, and $F \subseteq Q$ is the accepting set. The extended transition δ^* consumes a string.
- **ε-nondeterministic finite automaton (ε-NFA):** A machine permitting multiple successor states and transitions that consume no character. Its active configuration is a set of states; ε-closure(S) includes S and every state reachable from S using only ε-transitions.

$$L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\} \quad (1)$$

$$\varepsilon\text{-closure}(S) = \{q \mid \exists s \in S, s \to_{\varepsilon}^* q\} \quad (2)$$

B. Syntax and distinguishing inputs

- **Context-free grammar (CFG):** $G = (V, T, P, S)$, with nonterminals V, terminals T, productions P, and start symbol S.
- **Token:** A lexical category, source text, and interval [start, end), preserving the input location.
- **LL(1) parser:** A predictive parser that reads left to right, constructs a leftmost derivation, and selects a production using one lookahead token and the stack top.
- **Pushdown automaton (PDA):** A finite-control machine with a stack. AUTOSEC demonstrates this memory through pending symbols in its predictive parser.

TABLE II. NOTATION USED THROUGHOUT THE REPORT

Symbol	Meaning
Σ^*	All finite strings over Σ, including ε.
L(r), L(G)	Languages denoted by a regex and generated by the grammar.
$\delta^*(q, w)$	State reached after consuming w from state q.
$A \triangle B$	Symmetric difference: strings in exactly one language.
[s, e)	Start included; end excluded. Source offsets are zero-based.
$	End-of-input marker, not a character in the command.

- **Pattern and syntax:** Pattern search tests substring membership in L(r); syntax tests complete token-sequence membership in L(G).
- **Comparison:** Both regular rules use whole-string membership. A distinguishing string lies in their symmetric difference; a shortest witness minimises length and need not be unique.
- **Quoted input:** Raw matching may find `--tag` inside `inspect "--tag"`, while the lexer treats the quoted text as one value.

III. Methodology and System Architecture

A. Design and verification procedure

The methodology evaluates a reproducible input or rule pair through four stages:

1) **Specification:** Define the regular-expression subset, fixed command grammar, token categories, and recognition semantics.
2) **Construction:** Implement explicit automata, calculated grammar sets, predictive decisions, and execution traces.
3) **Integration:** Connect independent pattern and syntax pipelines through a small browser interface.
4) **Verification:** Check independent language predicates, exact product comparisons, malformed inputs, and the running interface. Related educational activity design informs the context [8].

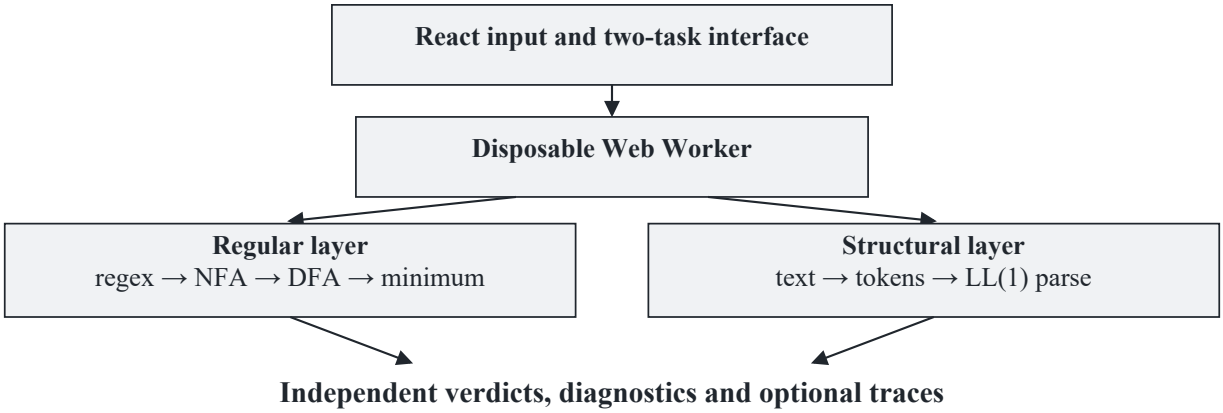


Fig. 1. Independent regular and structural pipelines inside a local worker.

B. Computation protocol

- **Interface:** React manages input and result state; Vite builds the TypeScript application.
- **Dispatch:** Form submission creates a disposable module Web Worker and sends `{id, task, data}`. The dispatcher calls `analyse()` or compiles both rules before `compareDfa()`.
- **Response and lifecycle:** The worker returns serialisable machines, tokens, traces, and diagnostics. Editing clears results; switching views disposes of pending computation. The algorithm engine remains independent of rendering.

TABLE III. METHODOLOGY AND CORRESPONDING ARTEFACT

Stage	Concrete procedure	Evidence
Specification	Fix regex subset, token categories and nested grammar.	Definitions and productions.
Construction	Generate automata and predictive decisions from inputs.	Machine edges, sets and traces.
Integration	Connect regular and structural engine results.	Worker responses and verdicts.
Verification	Check predicates, products and browser cases.	90 tests; browser captures.

C. Resource and operational boundaries

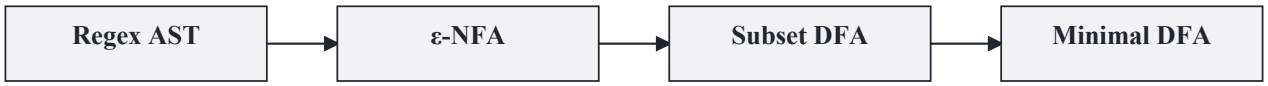
- **Resource limits:** 98 ASCII symbols; 128 regex characters; 2,048 input characters; 256 NFA states; 512 DFA states; and a 12-second request timeout.
- **Incomplete analysis:** Cancellation or resource exhaustion produces **analysis incomplete**, not rejection or equivalence. Unsupported constructs receive diagnostics.
- **Local operation:** Ordinary use requires no external service. Input is not sent to a backend, executed as commands, or used to visit entered URLs.

IV. Regular-Language Construction

A. Parsing and Thompson construction

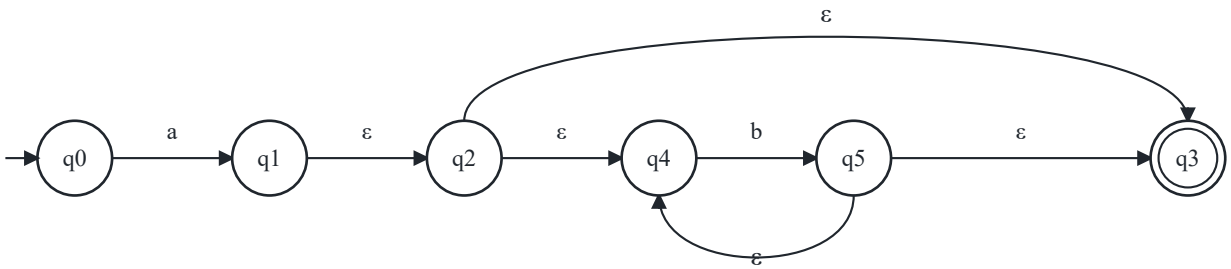
The regular-language pipeline follows these steps:

1) **Parse:** `parseRegex()` constructs an abstract syntax tree using alternation, concatenation, and repetition precedence.
2) **Construct:** `thompson()` builds entry/exit fragments. Literal edges consume symbols; concatenation, alternation, and repetition introduce the required ε-links, branches, and loops.
3) **Check the subset:** Character classes such as `[a-z]` expand into supported symbols. Backreferences, lookarounds, anchors, and counted repetitions are rejected.



B. Worked construction for ab*

- **Language:** `ab*` denotes `a` followed by zero or more `b` symbols. Fig. 2 uses the engine's recorded state IDs and edges.
- **Acceptance after a:** ε-closure reaches {1,2,3,4}, including accepting NFA state 3; therefore `a` is accepted.
- **Subset states:** Each reachable closed set becomes a DFA state. The empty subset represents a rejecting sink when reachable, as recorded in Table IV.



All stored transitions shown; double circle marks acceptance.

Fig. 2. Actual Thompson ε-NFA for ab*. State IDs and edges follow the worker fixture.

TABLE IV. REACHABLE DFA SUBSETS FOR ab*

DFA state	Represented NFA set	Accepting?
q0	{0}	No
q1	{1,2,3,4}	Yes
q2	{}	No
q3	{3,4,5}	Yes

C. Minimisation and implementation limits

- **Partition refinement:** Accepting and rejecting states are separated, then blocks with distinguishable transitions are split. DFA states 1 and 3 merge because they have the same future language.
- **Observed construction: 6 NFA → 4 DFA → 3 minimal-DFA states**. The Hopcroft-style worklist uses straightforward set scans; the optimised textbook running-time bound is not claimed.
- **Coverage:** Extended derivative-based regex research supports constructs outside this demo [9]. This is a scope comparison rather than a component dependency or a claim of full regex compatibility.

V. Lexical Analysis and Predictive Parsing

A. Fixed nested command language

- **Tokens:** The lexer recognises WORD, lowercase FLAG, double-quoted QUOTED values, `;`, `(`, and `)`. Quoted values permit escaped quotes and backslashes.
- **Constraints:** Adjacent words, flags, and quoted values require whitespace. Every flag requires a value; token offsets identify the original input span.
- **Failure boundary:** Lexing and parsing remain separate. A lexical failure prevents acceptance of a partial stream.

```
Script    → Statement Rest
Rest      → ; Statement Rest | ε
Statement → WORD Args | ( Script )
Args      → Arg Args | ε
Arg       → FLAG Value | Value
Value     → WORD | QUOTED
```

B. Calculated decisions

- **FIRST(A):** Terminals that can begin strings derived from A, including ε when A is nullable.
- **FOLLOW(A):** Terminals that can immediately follow A; `$` marks end-of-input.
- **Table construction:** `grammarInfo()` calculates the sets to a fixed point and detects duplicate table assignments. The grammar has no conflicts; an empty array represents the empty production.

TABLE V. FIRST/FOLLOW SETS COMPUTED FROM THE FIXED GRAMMAR

Nonterminal	FIRST	FOLLOW
Script	{(, WORD}	{$,)}
Rest	{;, ε}	{$,)}
Statement	{(, WORD}	{$,), ;}
Args	{FLAG, QUOTED, WORD, ε}	{$,), ;}
Arg	{FLAG, QUOTED, WORD}	{$,), ;, FLAG, QUOTED, WORD}
Value	{QUOTED, WORD}	{$,), ;, FLAG, QUOTED, WORD}

C. Dry run and stack interpretation

1) **Initial state:** For `inspect --tag sample; (scan file)`, the stack begins with Script over `$`. The worker records **8 tokens and 25 parser actions**.
2) **Expansion:** Right-hand sides are pushed in reverse order so the leftmost symbol is processed next. An opening group leaves a closing `)` pending until its inner Script finishes.

TABLE VI. FIRST FOUR ACTIONS FOR THE VALID COMMAND

Step	Stack top first	Applied action
1	Script $	Script → Statement Rest
2	Statement Rest $	Statement → WORD Args
3	WORD Args Rest $	Match WORD: "inspect"
4	Args Rest $	Args → Arg Args

- **Missing value:** `inspect --tag` leaves Value pending at offset 13 while the pattern still matches. Removing the valid example's final `)` leaves a closing delimiter pending at offset 32.
- **Verification scope:** Diagnostics retain zero-based offsets. POSIX lexing [10] and verified lexer work [11] address stronger proof obligations; this implementation is tested rather than mechanically verified.

VI. Shortest-Counterexample Method

A. Product construction

- **State pairs:** For generated DFAs A and B, a product node is (p, q). Each symbol advances both components; an undefined transition enters an implicit rejecting sink.
- **Disagreement condition:** A node distinguishes the languages exactly when one component accepts and the other rejects.
- **Search domain:** The search uses the union of both alphabets and visits each reachable pair once.

$$L(A) \triangle L(B) = (L(A) - L(B)) \cup (L(B) - L(A)) \quad (3)$$

$$|w^*| = \min \{|w| : w \in L(A) \triangle L(B)\} \quad (4)$$

```
queue ← [(startA, startB)]
for each pair in queue, in discovery order:
    if acceptingA(pair.left) XOR acceptingB(pair.right):
        return the input reconstructed through parent links
    for each symbol in the shared alphabet:
        enqueue the successor pair if it has not been seen
return equivalent
```

B. Shortestness argument and observed witness

1) **Shortestness:** Every edge consumes one character. Breadth-first search visits nondecreasing input lengths; the first acceptance-disagreeing node therefore has a shortest path.
2) **Predecessors:** One shortest predecessor per pair suffices because future acceptance depends on the pair, not its earlier path.
3) **Equivalence:** Exhausting the reachable product without disagreement establishes an empty symmetric difference. Interruption produces an incomplete result.



After --a: A accepts; B rejects. Only the witness path is drawn.

Fig. 3. Paired DFA run along --a. Numeric labels belong to separate machines.

TABLE VII. COMPUTED ACCEPTANCE ALONG THE SHORTEST WITNESS

Prefix	A state / acceptance	B state / acceptance
ε	q0 / reject	q0 / reject
-	q1 / reject	q1 / reject
--	q3 / reject	q3 / reject
--a	q4 / accept	q4 / reject

- **Observed witness:** A = `--[a-z]+` requires at least one lowercase letter after two hyphens; B = `--[a-z][a-z]+` requires at least two. The witness `--a` is found after **5 processed pairs**. Every shorter string is rejected by both rules; numeric state labels are local to each machine.
- **Complexity and scope:** Product search uses O(kmn) time and O(mn) storage for m and n completed DFA states and alphabet size k, excluding compilation. This exact comparison differs from AALpy's learning/oracle workflow [7].

VII. Actual Application Screenshots

- **Capture conditions:** The application ran at `http://127.0.0.1:4173/` with an 1100 × 900 desktop viewport. Full captures accompany the repository.
- **Evidence integrity:** Fig. 4 shows valid nested syntax; Fig. 5 isolates the missing-value diagnostic in a directly captured viewport excerpt. Results are unaltered; images are resized only for placement.

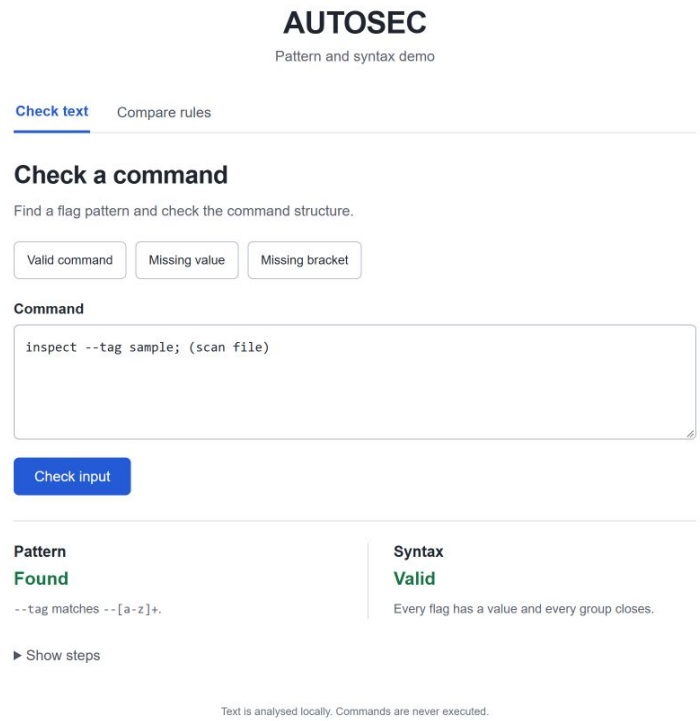


Fig. 4. Actual Check text view: the flag is found and the nested command is valid.

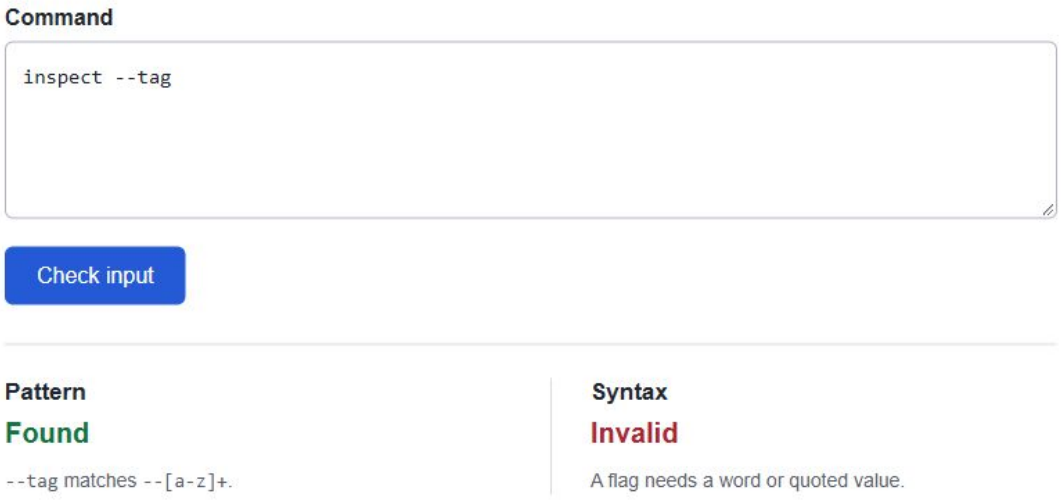


Fig. 5. Actual viewport excerpt: the flag remains present while a missing value makes syntax invalid.

- **Pattern result:** Both examples contain `--tag`; the regular pattern is found.
- **Syntax result:** Deleting the flag value changes acceptance to rejection. Pattern occurrence and complete-command validity are separate decisions.

VIII. Evaluation and Observed Results

A. Verification protocol

1) **Independent predicates:** Eight languages are evaluated on every string over {a,b} of length 0 through 6: 1 + 2 + 4 + 8 + 16 + 32 + 64 = 127 strings per language.
2) **Recognition checks:** Each expected answer is compared with NFA, DFA, and minimal-DFA simulation, producing **3,048 recognition checks** within the parameterised test cases.
3) **Complementary methods:** Exact product comparisons check language preservation; short-string enumeration alone does not prove behaviour for arbitrary strings. Engine and Document Object Model (DOM) interaction tests remain separate.

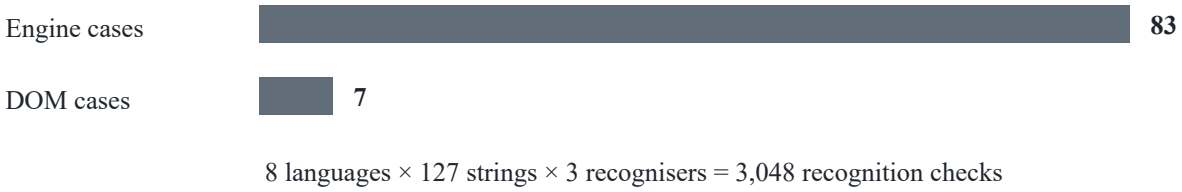


Fig. 6. Passing automated test cases and the embedded recognition-check count.

TABLE VIII. OBSERVED VERIFICATION RESULTS

Check	Observed result	Interpretation
Engine / DOM suite	83 / 7 passing	90 test cases; not 90 user participants.
Production build	Passed	TypeScript and Vite build successful.
ab* construction	6 → 4 → 3 states	NFA, DFA and minimal DFA.
Default comparison	--a; 5 processed pairs	A accepts and B rejects.
a+ versus aa*	Equivalent	Reachable product exhausted.
a* versus a+	Empty witness ε	Initial acceptance disagrees.

B. Behavioural cases and evidence provenance

TABLE IX. TWO INDEPENDENT OUTCOMES ON THE SAME INPUT

Input	Flag found?	Syntax accepted?
inspect --tag sample; (scan file)	Yes	Yes
inspect --tag	Yes	No
inspect file	No	Yes
(inspect file	No	No
inspect "--tag"	Yes	Yes

- **Browser cases:** Valid syntax, a missing value, the default witness and trace, and a missing bracket were reproduced. At 390px, document and viewport widths matched; no console warnings or errors were observed.
- **Additional checks:** A separate real-browser review verified cancellation, equivalence, an empty witness, and invalid-regex diagnostics. DOM cancellation tests also passed.
- **Evidence limits:** Built-worker fixtures check assets and serialisation, not browser timing. Threat-detection accuracy, learning gains, and comparative performance were not measured.

IX. Discussion, Limitations and Conclusion

A. Interpretation and contribution

- **Conceptual distinction:** A matching substring can occur in invalid syntax; a shortest witness explains disagreement between regular rules.
- **Role of nesting:** The non-nested command subset can be regular. Unrestricted nesting in the grammar motivates stack memory, while the implementation enforces finite resource bounds.
- **Contribution:** AUTOSEC integrates these observations in a compact workflow alongside the broader libraries [6], [7] and visualisation systems [2]-[4].

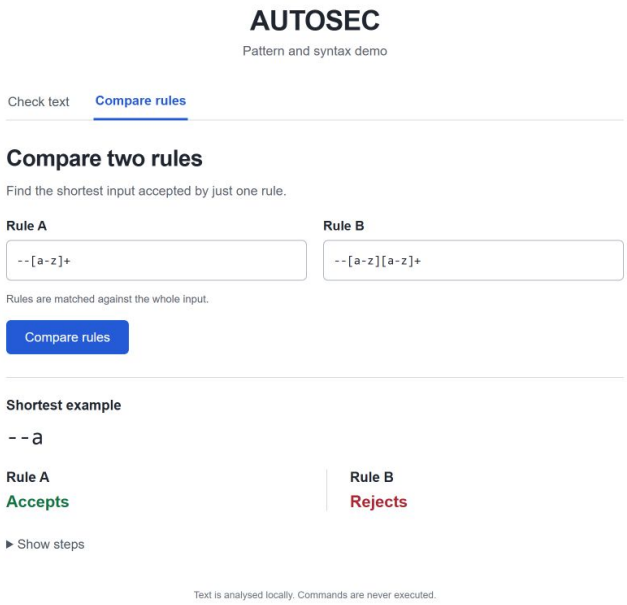


Fig. 7. Actual Compare rules view: --a is accepted only by Rule A.

B. Threats to validity and future evaluation

- **Selected examples:** Prepared demonstrations do not establish general usability. ASCII, token, regex, and state bounds restrict coverage.
- **Verification limits:** Some tests share infrastructure; independent predicates and exact product checks provide complementary evidence. Browser observations cover one browser and two viewport sizes.
- **Accessibility:** Labels and keyboard controls support inspection, but comprehensive Web Content Accessibility Guidelines (WCAG) 2.2 [12] compliance remains unassessed.
- **Future study:** Compare unaided and tool-assisted explanations, measuring witness reasoning, task time, and error rates. Educational activity design [8] and concise diagnostics [5] provide starting points for evaluating understanding.

C. Conclusion

AUTOSEC integrates automata construction, predictive parsing, and exact product search in a small browser application. Recorded tests, worked traces, and genuine screenshots support the reported behaviour. Its contribution is a reproducible route from an input or rule change to an explicit explanation. Educational effectiveness and comparative performance remain open evaluation questions.

References

[1] P. Linz and S. H. Rodger, *An Introduction to Formal Languages and Automata*, 7th ed. Burlington, MA, USA: Jones & Bartlett Learning, 2023. [Online]

[2] M. T. Morazán and T. Minić, "Finite-State Automaton To/From Regular Expression Visualization," *Electronic Proceedings in Theoretical Computer Science*, vol. 405, pp. 36-55, 2024. doi: 10.4204/EPTCS.405.3. [Online]

[3] O. Kempinski and M. T. Morazán, "Visualizing Why Nondeterministic Finite-State Automata Reject," *arXiv preprint arXiv:2310.08025*, presented at the Scheme and Functional Programming Workshop, 2023. doi: 10.48550/arXiv.2310.08025. [Online]

[4] M. T. Morazán, D. A. K. Fields, A. M. Garced, and T. Minić, "Visual Execution and Validation of Finite-State Machines and Pushdown Automata," *Electronic Proceedings in Theoretical Computer Science*, vol. 424, pp. 87-108, 2025. doi: 10.4204/EPTCS.424.5. [Online]

[5] M. T. Morazán et al., "A Design Recipe and Recipe-Based Errors for Regular Expressions," *Electronic Proceedings in Theoretical Computer Science*, vol. 424, pp. 25-48, 2025. doi: 10.4204/EPTCS.424.2. [Online]

[6] C. Evans and E. W. Robson, "automata: A Python package for simulating and manipulating automata," *Journal of Open Source Software*, vol. 8, no. 90, Art. no. 5759, 2023. doi: 10.21105/joss.05759. [Online]

[7] E. Muškardin, B. K. Aichernig, I. Pill, A. Pferscher, and M. Tappler, "AALpy: An active automata learning library," *Innovations in Systems and Software Engineering*, vol. 18, no. 3, pp. 417-426, 2022. doi: 10.1007/s11334-022-00449-3. [Online]

[8] J. Dochkova-Todorova and I. Donchev, "STEM Approach to Build Competences from Automata and Languages Theory," *Science Series Innovative STEM Education*, vol. 6, pp. 35-54, 2024. doi: 10.55630/STEM.2024.0604. [Online]

[9] I. E. Varatalu, M. Veanes, and J.-P. Ernits, "Derivative Based Extended Regular Expression Matching Supporting Intersection, Complement and Lookarounds," *arXiv preprint arXiv:2309.14401*, 2023. doi: 10.48550/arXiv.2309.14401. [Online]

[10] C. Tan and C. Urban, "POSIX Lexing with Bitcoded Derivatives," *Proc. 14th Int. Conf. Interactive Theorem Proving (LIPIcs)*, vol. 268, pp. 27:1-27:18, 2023. doi: 10.4230/LIPIcs.ITP.2023.27. [Online]

[11] D. Egolf, S. Lasser, and K. Fisher, "Verbatim++: Verified, Optimized, and Semantically Rich Lexing with Derivatives," *Proc. 11th ACM SIGPLAN Int. Conf. Certified Programs and Proofs*, pp. 27-39, 2022. doi: 10.1145/3497775.3503694. [Online]

[12] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.2," *W3C Recommendation*, Oct. 5, 2023. [Online]